

278. First Bad Version Easy

Acceptance: 46.7% | [LeetCode](#)

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have `n` versions `[1, 2, ..., n]` and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether `version` is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

```
**Input:** n = 5, bad = 4
**Output:** 4
**Explanation:**
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.
```

Example 2:

```
**Input:** n = 1, bad = 1
**Output:** 1
```

Constraints:

- `1 <= bad <= n <= 231 - 1`

Claude Opus Solution

First Bad Version - Solution

1. Intuition

The key insight is recognizing the **monotonic property** of the versions:

Version:	1	2	3	4	5
Status:	good	good	good	bad	bad
				^	
				first bad version	

Once a version is bad, **all subsequent versions are also bad**. This creates a sorted "boolean" sequence: `[False, False, ..., True, True]`. We're essentially searching for the **leftmost True** (the boundary between good and bad).

Whenever you have a sorted/monotonic search space and want to minimize API calls, **Binary Search** is the natural choice. Instead of checking every version linearly ($O(n)$), we can eliminate half the search space each step ($O(\log n)$).

2. Approach (Pseudocode)

- Set search boundaries: `left = 1`, `right = n`
- While `left < right`:
 - Compute `mid = left + (right - left) / 2` (avoid overflow)
 - If `isBadVersion(mid)` is True:
 - `mid` could be the first bad version, but earlier ones might also be bad $\rightarrow$ search LEFT half (include `mid`): `right = mid`
 - Else (`mid` is good):
 - First bad must be AFTER `mid` $\rightarrow$ search RIGHT half: `left = mid + 1`
- When `left == right`, we've converged to the first bad version $\rightarrow$ return `left`

Why `right = mid` (not `mid - 1`)? When `mid` is bad, it's a *candidate* answer, so we must keep it in the range.

Why `left = mid + 1`? When `mid` is good, it's definitely *not* the answer, so we safely discard it.

Why does the loop terminate? The range always shrinks, and we converge when `left == right` (a single element remains, which must be the answer).

3. Python Solution

```
# The isBadVersion API is defined as followed
def isBadVersion(version: int) -> bool:
    ...
```

```

class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Initialize the search space to all versions [1, n]
        left, right = 1, n

        # Continue narrowing until the boundary collapses to one element.
        # We use 'left < right' (not '<=') because we never need to check
        # when left == right (that single element IS our answer).
        while left < right:
            # Compute mid safely to avoid integer overflow in other languages.
            # In Python ints are unbounded, but this is good practice.
            mid = left + (right - left) // 2

            if isBadVersion(mid):
                # mid is bad -> first bad version is at mid OR to its left.
                # Keep mid in the search range as a possible answer.
                right = mid
            else:
                # mid is good -> first bad version must be strictly to the right.
                # Discard mid by moving left past it.
                left = mid + 1

        # Loop exits when left == right: both point to the first bad version.
        return left

```

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(\log n)$	Each iteration halves the search space. Starting with n candidates, we need at most $\lceil \log_2(n) \rceil$ iterations (and API calls).
Space	$O(1)$	Only two pointers (<code>left</code> , <code>right</code>) and <code>mid</code> are used — constant extra memory.

Minimizing API calls: Binary search makes $\sim \log_2(n)$ calls. For $n = 2^{31} - 1$, that's only **~31 calls** versus up to **2 billion** with a linear scan!

5. Alternative Approaches

A) Linear Search (Naive) — ❌ Too Slow

```

def firstBadVersion(self, n: int) -> int:
    for v in range(1, n + 1):
        if isBadVersion(v):
            return v

```

- **Time:** $O(n)$, **Space:** $O(1)$
- Violates the "minimize API calls" requirement; impractical for large n .

B) Binary Search with `left <= right` template

```

def firstBadVersion(self, n: int) -> int:
    left, right = 1, n
    result = n
    while left <= right:
        mid = left + (right - left) // 2
        if isBadVersion(mid):
            result = mid          # record candidate
            right = mid - 1      # try to find an earlier bad version
        else:
            left = mid + 1
    return result

```

- Same $O(\log n)$ time, but uses an extra `result` variable.
- Functionally equivalent; some prefer this for clarity of "tracking the best answer."

C) Using Python's `bisect` (Conceptual)

The problem is essentially `bisect_left` on a virtual boolean array. While you can't directly pass the API to `bisect`, the underlying algorithm is identical to our solution.

Key Takeaway

Pattern recognized: "Find the leftmost element satisfying a condition in a monotonic sequence" → **Binary Search on the answer space.**

This template (`right = mid` for the boundary case, `left = mid + 1` otherwise) is reusable for many problems like *Search Insert Position*, *Find Minimum in Rotated Sorted Array*, and *Find First/Last Position in Sorted Array*.

Codex Solution

1. Intuition

This problem has a **monotonic pattern**:

Good Good Good Bad Bad Bad Bad

Once a version is bad, every version after it is also bad.

So we are looking for the **first True** in a sorted boolean-like sequence:

False False False True True True

This is a classic **binary search on answer** problem.

Instead of checking every version one by one, we can repeatedly cut the search space in half:

- If `mid` is bad, the first bad version is either `mid` or somewhere before it.
- If `mid` is good, the first bad version must be after `mid`.

2. Approach

We binary search over versions from `1` to `n`.

Pseudocode

```
left = 1
right = n

while left < right:
    mid = middle of left and right

    if isBadVersion(mid) is True:
        right = mid
        # mid might be the first bad version,
        # so we keep it in the search range

    else:
        left = mid + 1
        # mid is good, so first bad must be after mid

return left
```

At the end, `left == right`, and that position is the first bad version.

3. Python Solution

```
# The isBadVersion API is already provided by LeetCode.
# You should not implement it yourself.
# def isBadVersion(version: int) -> bool:
#     ...

class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Search range is from version 1 to version n
        left = 1
        right = n
```

```

# Continue until the search range collapses to one version
while left < right:
    # Find the middle version.
    #
    # In Python, (left + right) // 2 is safe because integers
    # do not overflow. In languages like Java/C++, prefer:
    # mid = left + (right - left) // 2
    mid = (left + right) // 2

    # If mid is bad, then the first bad version could be mid
    # or some version before mid.
    if isBadVersion(mid):
        right = mid

    # If mid is good, then every version up to mid is good.
    # Therefore, the first bad version must be after mid.
    else:
        left = mid + 1

# At this point, left == right.
# This is the smallest version that is bad.
return left

```

4. Complexity Analysis

Let n be the total number of versions.

Time Complexity

$O(\log n)$

Each call to `isBadVersion` eliminates half of the remaining search space.

For example, if there are n versions:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

So the number of API calls is logarithmic.

Space Complexity

$O(1)$

We only use a few variables:

`left, right, mid`

No extra data structures are used.

5. Alternative Approaches

1. Linear Search

We could check versions from 1 to n :

```

for version in range(1, n + 1):
    if isBadVersion(version):
        return version

```

However, this takes:

$O(n)$

This is inefficient when n is large.

2. Binary Search

The optimal approach is binary search because the versions follow a monotonic pattern:

Good ... Good Bad ... Bad

Binary search reduces API calls to:

$O(\log n)$

This satisfies the requirement to minimize calls to `isBadVersion`.